# Studio Nx

AI 구현 설명 08 — 백엔드 공통 뼈대

2026-10-02

# AI 구현 설명 08 — 백엔드 공통 뼈대

| 항목 | 내용 |
|---|---|
| 작성 | Claude (AI) |
| 작성일 | 2026-10-02 |
| 작업 | P1-03 백엔드 공통: ORM 모델, 오류 응답 형식, `get_current_user` (**뼈대**). 이전 문서: 07_auth_pages.md |
| 브랜치 | `feature/phase1/p1-03-backend-core` |
| 원칙 | 백엔드는 **템플릿만** — 클래스·함수 이름, 형식, TODO. 실제 구현은 사용자 |

---

## 1. 무엇을 만들었나

| 파일 | 상태 | 내용 |
|---|---|---|
| `app/models/user.py` | 뼈대 | `User` — 테이블 이름, 기본 키. 나머지 컬럼은 TODO 목록 |
| `app/models/inquiry.py` | 뼈대 | `Inquiry` |
| `app/models/project.py` | 뼈대 | `Project`, `ProjectMember`(복합 기본 키 `project_id` + `user_id`) |
| `app/models/track.py` | 뼈대 | `Track` — Phase 1에 필요한 컬럼(MASTER 트랙 생성용)과 Phase 2 컬럼을 나눠 TODO |
| `app/models/__init__.py` | 완성 | 모델 모음 (관계 문자열 이름 연결용) |
| `app/schemas/common.py` | 완성 | `ErrorResponse` — 모든 오류 응답의 모양 |
| `app/core/errors.py` | 뼈대 | `ErrorCode`(오류 코드 목록), `AppError`(완성), `register_exception_handlers`(TODO) |
| `app/core/security.py` | 뼈대 | 비밀번호 해시, 토큰 발급·검사 함수 이름과 TODO (P1-03·04·05에서 채움) |
| `app/api/deps.py` | 뼈대 | `get_current_user`, `get_current_user_optional`, 별칭 `CurrentUser`, `OptionalUser` |
| `app/main.py` | 수정 | `register_exception_handlers(app)` 호출 |
| `tests/test_models.py` | 완성 | 모델의 테이블 이름·기본 키가 SQL과 같은지 확인 |

## 2. 왜 이렇게 했나

### 2.1 오류 응답 형식

모든 API가 같은 모양으로 오류를 돌려주면, 프론트(P1-02)의 API 함수 하나가 모든 오류를 같은 방법으로 처리할 수 있습니다.

```
{"error": {"code": "EMAIL_TAKEN", "message": "This email is already in use.", "details": null}}
```

| 키 | 용도 |
|---|---|
| `code` | 프론트가 분기에 쓰는 고정 문자열. 문장이 바뀌어도 코드는 그대로 |
| `message` | 화면에 보여줄 영어 문장 |
| `details` | 입력값 오류일 때 칸별 문구 (예: `{"nickname": "2–20 characters"}`) |

미리 정한 오류 코드(`ErrorCode`)는 다음과 같습니다.

| 코드 | 상태 | 쓰는 곳 |
|---|---|---|
| `VALIDATION_ERROR` | 422 | 입력값 형식 오류 |
| `UNAUTHORIZED` | 401 | 토큰 없음·만료 → 프론트가 재발급 시도 |
| `INVALID_CREDENTIALS` | 401 | 로그인 실패 (FR-103 통일 문구) |
| `FORBIDDEN` | 403 | 다른 사람 프로젝트 등 |
| `NOT_FOUND` | 404 | — |
| `EMAIL_TAKEN`, `NICKNAME_TAKEN` | 409 | 회원가입 (P1-04) |
| `ACCOUNT_DISABLED` | 403 | 탈퇴·정지 계정 (FR-108) |
| `TOO_MANY_REQUESTS` | 429 | 문의 반복 전송 (FR-113) |
| `INTERNAL_ERROR` | 500 | 그 외. 자세한 내용은 응답에 넣지 않고 Sentry로만 |

### 2.2 `get_current_user`

로그인이 필요한 API는 인자에 `user: CurrentUser`만 쓰면 됩니다. 순서는 TODO에 적어 두었습니다.

```
Authorization: Bearer <access token>
 → decode_access_token (서명·만료·type 검사)
 → db.get(User, user_id)
 → status != 'ACTIVE' 이면 401 (탈퇴 계정 차단)
 → User 반환
```

- `HTTPBearer(auto_error=False)`: 헤더가 없을 때 FastAPI 기본 403 대신, 직접 401 + `UNAUTHORIZED`를 돌려주기 위해서입니다. 프론트는 401일 때만 재발급을 시도합니다.
- `Annotated` 방식(`CurrentUser = Annotated[User, Depends(...)]`)은 FastAPI가 권장하는 쓰는 법이고, 린트 규칙(B008)도 통과합니다.
- `get_current_user_optional`은 문의 등록처럼 로그인이 선택인 API용입니다 (FR-112: 로그인 상태면 `user_id` 연결).

### 2.3 모델은 컬럼 목록을 TODO로

모델 클래스·테이블 이름·기본 키만 만들고, 나머지 컬럼은 SQL과 같은 순서로 TODO에 적었습니다. 직접 `mapped_column`으로 옮기면서 SQLAlchemy 2 문법을 익히도록 하기 위해서입니다.

- 기본 키만 있어도 모델은 동작합니다. 실제 MySQL 테이블에서 5개 모델 모두 조회되는 것을 확인했습니다(§3).
- 기본값·자동 갱신(`created_at` 등)은 DB가 처리하므로 `server_default`로 표시만 하면 됩니다.

## 3. 검증 결과 (2026-10-02)

| 확인 | 결과 |
|---|---|
| `pytest` | 3개 통과 (health 1 + 모델 2) |
| `ruff check` | 통과 |
| 실제 DB 조회 | `select(User)` 등 5개 모델 모두 오류 없음 (테이블이 비어 있어 결과는 없음) |
| 서버 시작 | `/api/v1/health` 정상, `/docs` 200 |

## 4. 남은 일 (사용자)

| 할 일 | 파일 | 관련 작업 |
|---|---|---|
| 모델 나머지 컬럼·관계 채우기 | `app/models/*.py` | P1-03 |
| 오류 처리기 4종 등록 | `app/core/errors.py` | P1-03 |
| `decode_access_token`, `get_current_user` 구현 | `app/core/security.py`, `app/api/deps.py` | P1-03 |
| 비밀번호 해시 | `hash_password` | P1-04 |
| 토큰 발급, 비밀번호 확인 | `create_*_token`, `verify_password` | P1-05 |

> PyJWT(`jwt`)와 `bcrypt`는 `requirements.txt`에 이미 있습니다.

## 5. 자주 받을 질문 대비

- **Q. 왜 `HTTPException` 대신 `AppError`를 쓰나?** `HTTPException`은 `detail` 하나만 담습니다. `AppError`는 `code`·`message`·`details`를 나눠 담아, 오류 처리기가 항상 같은 모양으로 바꿔 줍니다.
- **Q. 422 입력값 오류도 같은 모양이 되나?** `register_exception_handlers`에서 `RequestValidationError`도 잡아 `VALIDATION_ERROR`로 바꾸도록 TODO에 적어 두었습니다.
- **Q. 모델로 테이블을 만들지 않나?** 테이블은 `studionx/db/*.sql`로 만듭니다(DB.md가 원본). 모델은 이미 있는 테이블을 읽고 쓰는 용도라 `Base.metadata.create_all()`은 쓰지 않습니다.